# Oracle 19c Daily Check health

## A) Database posture & uptime

### Role & protection posture

**Purpose:** Confirm Primary/Standby role, open status, logging mode, Flashback, and Data Guard protection.

**Risk:** OPEN_MODE != 'READ WRITE' on primary or LOG_MODE != 'ARCHIVELOG' is critical for recoverability.

**Notes:** Requires DBA/SELECT_CATALOG_ROLE.

SQL

```
SELECT database_role AS role, name, db_unique_name, open_mode, log_mode,
       flashback_on, protection_mode, protection_level, cdb
FROM   v$database;
```

**Status:**

**Comment:**

### Instance uptime

**Purpose:** Check uptime and version quickly.

**Risk:** Frequent restarts may indicate instability.

SQL

```
SELECT instance_name, host_name, version,
       FLOOR(SYSDATE - STARTUP_TIME) AS days,
       FLOOR(MOD((SYSDATE - STARTUP_TIME) * 24, 24)) AS hours,
       FLOOR(MOD((SYSDATE - STARTUP_TIME) * 24 * 60, 60)) AS minutes
FROM   v$instance;
```

**Status:**

**Comment:**

## B) Sessions & activity

### Session count by status

**Purpose:** Spot piling ACTIVE/INACTIVE states across RAC instances.

**Risk:** Large ACTIVE pile-ups plus high waits indicate bottlenecks.

Notes: Single-instance view using V$SESSION.

SQL

```
SELECT status, COUNT(*) AS sessions
FROM   v$session
GROUP  BY status
ORDER  BY status;
```

**Comment:**

## Long-running active sessions (non-background)

**Purpose:** Find hot/long SQL currently running.

**Risk:** Many entries with large elapsed_seconds is critical under user complaints.

SQL

```
SELECT s.sid, s.serial#, s.username, s.sql_id, s.last_call_et AS
elapsed_seconds,
       SUBSTR(q.sql_text, 1, 100) AS sql_text_sample
FROM   v$session s
JOIN   v$sql     q ON s.sql_id = q.sql_id
WHERE  s.status = 'ACTIVE'
AND    s.type   <> 'BACKGROUND'
AND    s.last_call_et > 60
ORDER  BY s.last_call_et DESC;
```

**Status:**

**Comment:**

## C) Multitenant (CDB/PDB) status & quotas

## PDB state & size

**Purpose:** Visibility of PDBs, open mode, and size.

**Risk:** Closed or restricted PDBs unexpectedly can affect apps.

SQL

```
SELECT con_id, name, open_mode, restricted, open_time, total_size
FROM   v$pdbs
ORDER  BY con_id;
```

**Status:**

**Comment:**

## PDB allocated/used/free across data & temp

**Purpose:** Consolidated view of allocated/used/free per PDB.

**Risk:** used_pct > 85% warning; > 95% critical.

**Notes:** CDB_FREE_SPACE doesn't reflect TEMP free; use V$TEMP_SPACE_HEADER for temp usage.

SQL

```
SET PAGES 999 LINES 200
SELECT p.name AS pdb_name,
       p.open_mode,
       p.restricted,
       ROUND(SUM(fs.bytes)/1024/1024/1024, 2)                  AS
allocated_size_gb,
       ROUND(SUM(fs.bytes - NVL(f.bytes, 0))/1024/1024/1024,2) AS used_size_gb,
       ROUND(SUM(NVL(f.bytes, 0))/1024/1024/1024, 2)           AS free_size_gb,
       ROUND((SUM(fs.bytes - NVL(f.bytes, 0)) / SUM(fs.bytes)) * 100, 2) AS
used_pct
FROM (
  SELECT con_id, file_id, tablespace_name, bytes FROM cdb_data_files
  UNION ALL
  SELECT con_id, file_id, tablespace_name, bytes FROM cdb_temp_files
) fs
LEFT JOIN (
  SELECT con_id, file_id, SUM(bytes) bytes
  FROM   cdb_free_space
  GROUP  BY con_id, file_id
) f ON fs.con_id = f.con_id AND fs.file_id = f.file_id
JOIN v$pdbs p ON fs.con_id = p.con_id
GROUP BY p.name, p.open_mode, p.restricted
ORDER BY used_pct DESC;
```

**Status:**

**Comment:**

## Tablespace quotas per PDB

**Purpose:** See quota ceilings used by PDB schemas/tablespaces.

**Risk:** Hitting quota limits will fail DML/segment growth.

**Notes:** Only shows where quotas are set (> 0).

SQL

```
SELECT p.name AS pdb_name,
       ROUND(tq.space_limit/1024/1024/1024, 2) AS space_limit_gb
FROM   cdb_ts_quotas tq
JOIN   v$pdbs p ON tq.con_id = p.con_id
WHERE  tq.space_limit > 0
ORDER  BY tq.space_limit DESC;
```

**Status:**

**Comment:**

## D) Space & growth

### CDB tablespace usage with max and autoextend

**Purpose:** Understand headroom vs MAXBYTES and autoextend.

**Risk:** used_pct_of_max > 85% warning; > 95% critical.

#### SQL

```
SELECT cdb.tablespace_name,
       c.name AS con_name,
       ROUND((cdb.bytes - SUM(fs.bytes)) * 100 / cdb.bytes, 2) AS usage_pct,
       CASE WHEN cdb.bytes    >= 1024*1024*1000 THEN
ROUND(cdb.bytes/1024/1024/1024,2)||' GB'
            ELSE ROUND(cdb.bytes/1024/1024,2)||' MB' END AS ts_size,
       CASE WHEN SUM(fs.bytes) >= 1024*1024*1000 THEN
ROUND(SUM(fs.bytes)/1024/1024/1024,2)||' GB'
            ELSE ROUND(SUM(fs.bytes)/1024/1024,2)||' MB' END AS ts_free,
       CASE WHEN (cdb.bytes - SUM(fs.bytes)) >= 1024*1024*1000
            THEN ROUND((cdb.bytes - SUM(fs.bytes))/1024/1024/1024,2)||' GB'
            ELSE ROUND((cdb.bytes - SUM(fs.bytes))/1024/1024,2)||' MB' END AS
used_ts,
       ROUND(SUM(fs.bytes) * 100 / cdb.bytes, 2)               AS free_pct,
       CASE WHEN cdb.maxbytes >= 1024*1024*1000 THEN
ROUND(cdb.maxbytes/1024/1024/1024,2)||' GB'
            ELSE ROUND(cdb.maxbytes/1024/1024,2)||' MB' END    AS max_size,
       ROUND((cdb.bytes - SUM(fs.bytes))/NULLIF(cdb.maxbytes,0)*100, 2) AS
used_pct_of_max,
       MAX(cdb.autoextensible) AS auto_ext
FROM   cdb_free_space fs
JOIN (
  SELECT con_id, tablespace_name,
         SUM(bytes) bytes,
         SUM(DECODE(maxbytes,0,bytes,maxbytes)) maxbytes,
         MAX(autoextensible) autoextensible
  FROM   cdb_data_files
  GROUP  BY con_id, tablespace_name
) cdb
  ON fs.con_id = cdb.con_id AND fs.tablespace_name = cdb.tablespace_name
JOIN v$containers c ON c.con_id = cdb.con_id
GROUP  BY cdb.tablespace_name, cdb.bytes, cdb.maxbytes, c.name
ORDER  BY 2 DESC;
```

**Status:**

**Comment:**

### Non-CDB/root quick usage

**Purpose:** Fast usage snapshot for non-CDB or root.

**Risk:** High usage_pct risks space errors.

SQL

```
SELECT df.tablespace_name,
       ROUND((df.bytes - fs.free_bytes) * 100 / df.bytes, 2) AS usage_pct,
       ROUND(df.bytes/1024/1024,2) AS ts_size_mb,
       ROUND(fs.free_bytes/1024/1024,2) AS ts_free_mb,
       ROUND((df.bytes - fs.free_bytes)/1024/1024,2) AS used_mb
FROM  (SELECT tablespace_name, SUM(bytes) bytes FROM dba_data_files GROUP BY
tablespace_name) df
LEFT  JOIN (SELECT tablespace_name, SUM(bytes) free_bytes FROM dba_free_space
GROUP BY tablespace_name) fs
ON    df.tablespace_name = fs.tablespace_name
ORDER BY usage_pct DESC;
```

**Status:**

**Comment:**

## Metrics view (fast)

**Purpose:** Quick high-level tablespace usage.

**Risk:** High used_percent indicates limited headroom.

SQL

```
SELECT tablespace_name,
       ROUND(used_space/1024/1024,2) used_mb,
       ROUND(tablespace_size/1024/1024,2) total_mb,
       ROUND(used_percent,2) used_pct,
       ROUND((tablespace_size - used_space)/1024/1024,2) free_mb
FROM   dba_tablespace_usage_metrics
ORDER  BY used_percent DESC;
```

**Status:**

**Comment:**

## Autoextend datafiles

**Purpose:** Know where growth will (or won't) happen.

**Risk:** No autoextend and high usage ⇒ growth failures.

SQL

```
SELECT tablespace_name, file_name, autoextensible,
       ROUND(bytes/1024/1024,2)    AS current_size_mb,
       ROUND(maxbytes/1024/1024,2) AS max_size_mb
FROM   dba_data_files
WHERE  autoextensible = 'YES'
ORDER  BY tablespace_name, file_name;
```

**Status:**

**Comment:**

## TEMP usage

**Purpose:** Track temporary space usage.

**Risk:** Free < 5–10% under load is critical.

### SQL

```
SELECT * FROM v$temp_space_header;
```

**Status:**

**Comment:**

## UNDO size & status

**Purpose:** Check UNDO sizing and status.

**Risk:** Frequent ORA-01555 suggests increasing UNDO/retention.

### SQL

```
SELECT ts.tablespace_name, ts.status,
       ROUND(SUM(df.bytes)/1024/1024,2) size_mb
FROM   dba_tablespaces ts
JOIN   dba_data_files df ON ts.tablespace_name = df.tablespace_name
WHERE  ts.contents = 'UNDO'
GROUP  BY ts.tablespace_name, ts.status;
```

**Status:**

**Comment:**

## Free space fragmentation

**Purpose:** Identify fragmentation; consider coalescing.

**Risk:** Many small chunks can cause allocation issues.

### SQL

```
SELECT tablespace_name, COUNT(*) AS free_chunks,
       ROUND(SUM(bytes)/1024/1024, 2) AS free_mb
FROM   dba_free_space
GROUP  BY tablespace_name
ORDER  BY free_mb DESC;
```

**Status:**

**Comment:**

## E) Security & users

### Users nearing expiry / locked (CDB)

**Purpose:** Catch expiring or locked accounts proactively.

**Risk:** Dormant unlocked accounts are critical risk.

SQL

```
SELECT u.con_id,
       (SELECT name FROM v$pdbs WHERE con_id = u.con_id) AS pdb_name,
       u.username, u.account_status, u.expiry_date
FROM   cdb_users u
WHERE  ORACLE_MAINTAINED = 'N'
AND   (u.account_status IN ('LOCKED','EXPIRED')
       OR (u.expiry_date IS NOT NULL AND u.expiry_date < SYSDATE + 28))
AND   (u.expiry_date IS NULL OR u.expiry_date > ADD_MONTHS(SYSDATE, -2))
ORDER  BY u.con_id, u.expiry_date DESC;
```

**Status:**

**Comment:**

### Users nearing expiry / locked (non-CDB/root)

**Purpose:** Same as above for non-CDB/root.

**Risk:** As above.

SQL

```
SELECT NULL AS con_id, NULL AS pdb_name, username, account_status, expiry_date
FROM   dba_users
WHERE  ORACLE_MAINTAINED = 'N'
AND   (account_status IN ('LOCKED','EXPIRED')
       OR (expiry_date IS NOT NULL AND expiry_date < SYSDATE + 28))
AND   (expiry_date IS NULL OR expiry_date > ADD_MONTHS(SYSDATE, -2))
ORDER  BY expiry_date DESC NULLS LAST;
```

**Status:**

**Comment:**

### Recent failed logins (classic audit)

**Purpose:** Detect brute force / misconfigurations.

**Risk:** High failed_attempts needs immediate review.

**Notes:** For Unified Auditing, switch to UNIFIED_AUDIT_TRAIL.

SQL

```
SELECT username, COUNT(*) AS failed_attempts, MAX(timestamp) AS last_failed
FROM   dba_audit_trail
WHERE  returncode != 0
```

```
AND    timestamp > SYSDATE - 7
GROUP  BY username
ORDER  BY failed_attempts DESC;
```

**Status:**

**Comment:**

## Invalid objects (non-SYS)

**Purpose:** Find invalids after patching/deploy.

**Risk:** Many invalids can break code paths.

### SQL

```
SELECT owner, object_type, COUNT(*) AS invalid_count
FROM   dba_objects
WHERE  status = 'INVALID'
AND    owner NOT IN ('SYS','SYSTEM','OUTLN','DBSNMP')
GROUP  BY owner, object_type
ORDER  BY invalid_count DESC;
```

**Status:**

**Comment:**

## DB links inventory

**Purpose:** Know external connectivity points.

**Risk:** Unknown links may enable data exfiltration.

### SQL

```
SELECT db_link, username, host, created
FROM   dba_db_links
ORDER  BY created DESC;
```

**Status:**

**Comment:**

# F) Performance (SQL, waits, PGA)

## Top CPU SQL (exclude SYS)

**Purpose:** Locate most CPU-expensive SQL.

**Risk:** High CPU with poor plans requires tuning.

### SQL

```
SELECT sql_id, executions,
       ROUND(cpu_time/1e6, 2)     AS cpu_seconds,
       ROUND(elapsed_time/1e6, 2) AS elapsed_seconds,
       SUBSTR(sql_text, 1, 100)   AS sql_text_sample
```

```
FROM   v$sql
WHERE  parsing_schema_name NOT IN ('SYS','SYSTEM')
ORDER  BY cpu_time DESC
FETCH FIRST 10 ROWS ONLY;
```

**Status:**

**Comment:**

## Top IO (disk reads)

**Purpose:** Find IO-heavy SQL.

**Risk:** High reads_per_exec suggests missing indexes or poor joins.

### SQL

```
SELECT sql_id, disk_reads, buffer_gets, executions,
       ROUND(disk_reads/NULLIF(executions,0), 2) AS reads_per_exec,
       SUBSTR(sql_text, 1, 100) AS sql_text_sample
FROM   v$sql
WHERE  parsing_schema_name NOT IN ('SYS','SYSTEM')
ORDER  BY disk_reads DESC
FETCH FIRST 10 ROWS ONLY;
```

**Status:**

**Comment:**

## System wait profile (non-idle)

**Purpose:** Identify most time-consuming waits.

**Risk:** High waits in User I/O / Concurrency may be critical.

### SQL

```
SELECT event, total_waits,
       ROUND(time_waited_micro/1e6, 2) AS time_waited_seconds,
       ROUND(average_wait/1000, 2)     AS average_wait_ms
FROM   v$system_event
WHERE  wait_class <> 'Idle'
ORDER  BY time_waited_micro DESC
FETCH FIRST 15 ROWS ONLY;
```

**Status:**

**Comment:**

## Aggregate wait classes

**Purpose:** High-level wait mix.

**Risk:** Dominant wait class guides next actions.

SQL

```
SELECT wait_class, ROUND(SUM(time_waited_micro)/1e6,1) AS seconds
FROM   v$system_event
WHERE  wait_class <> 'Idle'
GROUP  BY wait_class
ORDER  BY seconds DESC;
```

**Status:**

**Comment:**

## PGA health

**Purpose:** Check memory pressure for work areas.

**Risk:** over allocation count > 0 is critical.

SQL

```
SELECT name, value
FROM   v$pgastat
WHERE  name IN ('total PGA allocated','total PGA inuse','over allocation
count')
ORDER  BY name;
```

**Status:**

**Comment:**

## G) Locks & scheduler

## Blocking locks map

**Purpose:** Quickly identify blockers and victims.

**Risk:** Persistent chains are critical.

SQL

```
SELECT l1.sid AS blocking_sid, s1.username AS blocking_user,
       l2.sid AS blocked_sid,  s2.username AS blocked_user,
       l1.type AS lock_type
FROM   v$lock l1
JOIN   v$lock l2 ON l1.id1 = l2.id1 AND l1.id2 = l2.id2
JOIN   v$session s1 ON l1.sid = s1.sid
JOIN   v$session s2 ON l2.sid = s2.sid
WHERE  l1.block > 0 AND l2.request > 0;
```

**Status:**

**Comment:**

### Scheduler jobs with issues

**Purpose:** Find failing jobs that may affect housekeeping.

**Risk:** Backups/log purges failing is critical.

#### SQL

```
SELECT j.job_name, j.state, j.last_start_date, j.next_run_date, j.failure_count
FROM   dba_scheduler_jobs j
WHERE  j.state != 'SUCCEEDED' OR j.failure_count > 0
ORDER  BY j.failure_count DESC, j.job_name;
```

**Status:**

**Comment:**

## H) Data Guard / redo / archive  (ON standby)

### Transport vs apply gap (sequence diff)

**Purpose:** Simple indicator of lag.

**Risk:** Persistent diff > 0 under normal load is warning/critical.

**Notes:** Prefer V$DATAGUARD_STATS for authoritative lag.

#### SQL

```
SELECT ARCH.thread#,
       ARCH.sequence# AS last_seq_received,
       APPL.sequence# AS last_seq_applied,
       (ARCH.sequence# - APPL.sequence#) AS diff
FROM  (SELECT thread#, MAX(sequence#) KEEP (DENSE_RANK LAST ORDER BY
first_time) AS sequence#
       FROM   v$archived_log
       GROUP  BY thread#) ARCH
JOIN  (SELECT thread#, MAX(sequence#) KEEP (DENSE_RANK LAST ORDER BY
first_time) AS sequence#
       FROM   v$log_history
       GROUP  BY thread#) APPL
  ON  ARCH.thread# = APPL.thread#
ORDER BY 1;
```

**Status:**

**Comment:**

## DG lag stats (ON standby)

**Purpose:** Authoritative Data Guard lags and finish time.

**Risk:** Transport/apply lag beyond RPO is critical.

SQL

```
SELECT name, value, unit, time_computed
FROM   v$dataguard_stats
WHERE  name IN ('apply lag','transport lag','apply finish time')
ORDER  BY name;
```

**Status:**

**Comment:**

### Archive destinations health

**Purpose:** Monitor FRA/DEST usage and status.

**Risk:** Near-full FRA will stall logging (critical).

SQL

```
SELECT destination, status, archived,
       ROUND(NVL(used_mb,0)/1024, 2) AS used_gb
FROM   v$archive_dest_status
WHERE  status <> 'INACTIVE';
```

**Status:**

**Comment:**

### Redo logs (size/state)

**Purpose:** Check redo group sizing and health.

**Risk:** Very frequent switches imply too-small redo.

SQL

```
SELECT group#, thread#, sequence#, bytes/1024/1024 AS size_mb, status, members
FROM   v$log
ORDER  BY thread#, group#;
```

**Status:**

**Comment:**

### Hourly log switches (last hour)

**Purpose:** See churn within the last hour.

**Risk:** Too many switches may cause checkpoints/stalls.

#### SQL

```
SELECT TO_CHAR(first_time,'YYYY-MM-DD HH24') AS hr,
       COUNT(*) AS switches
FROM   v$log_history
WHERE  first_time > SYSDATE - 1/24
GROUP  BY TO_CHAR(first_time,'YYYY-MM-DD HH24')
ORDER  BY hr DESC;
```

**Status:**

**Comment:**

## I) Backups & patching

### RMAN jobs (10 days)

**Purpose:** Verify backup currency and duration.

**Risk:** No successful full/inc in policy window is critical.

#### SQL

```
SELECT session_key, input_type, status,
       TO_CHAR(start_time,'mm/dd/yy hh24:mi') AS start_time,
       TO_CHAR(end_time,'mm/dd/yy hh24:mi')   AS end_time,
       elapsed_seconds/3600 AS hrs
FROM   v$rman_backup_job_details
WHERE  start_time > SYSDATE - 10
ORDER  BY session_key DESC;
```

**Status:**

**Comment:**

### SQL patches registry

**Purpose:** Confirm post-patching state.

**Risk:** Failed or missing patches can cause issues.

#### SQL

```
SELECT patch_id, version, action, status, description
FROM   dba_registry_sqlpatch
ORDER  BY patch_id DESC;
```

**Status:**

**Comment:**

## J) Parameters & audit segment

### Key memory/limits

**Purpose:** Quick look at critical parameters.

**Risk:** Too small processes/open_cursors causes errors.

SQL

```
SELECT name, value
FROM   v$parameter
WHERE  name IN ('memory_target','processes','open_cursors','db_block_size',

'sga_target','pga_aggregate_target','sga_max_size','session_cached_cursors')
ORDER  BY name;
```

**Status:**

**Comment:**

### Audit segment size (classic audit)

**Purpose:** Track growth of AUD* tables.

**Risk:** Uncontrolled growth can bloat tablespace.

**Notes:** Unified Auditing stores in SYSAUX, monitor SYSAUX.

SQL

```
SELECT tablespace_name, ROUND(SUM(bytes)/1024/1024, 2) AS size_mb
FROM   dba_segments
WHERE  segment_name LIKE 'AUD%'
GROUP  BY tablespace_name;
```

**Status:**

**Comment:**

## K) AWR/ASH/ADDM

### ADDM findings (last 7 days)

**Purpose:** Summarize findings for quick triage.

**Risk:** High impact findings need action.

**Notes:** Diagnostic & Tuning Pack required.

### SQL

```
WITH tasks AS (
  SELECT task_id, task_name, instance_id AS instance_number, execution_start,
execution_end
  FROM   dba_advisor_tasks
  WHERE  advisor_name = 'ADDM'
  AND    status = 'COMPLETED'
  AND    execution_end > SYSDATE - 7
)
SELECT t.instance_number, t.task_name,
       TO_CHAR(t.execution_start,'YYYY-MM-DD HH24:MI') AS start_time,
       TO_CHAR(t.execution_end  ,'YYYY-MM-DD HH24:MI') AS end_time,
       f.type AS finding_type, f.impact_type,
       ROUND(f.impact,2) AS impact,
       SUBSTR(f.message,1,200) AS message_sample
FROM   tasks t
JOIN   dba_advisor_findings f ON f.task_id = t.task_id
ORDER  BY t.execution_start DESC, impact DESC
FETCH FIRST 60 ROWS ONLY;
```

**Status:**

**Comment:**

## ADDM recommendations

**Purpose:** Actionable recommendations and benefits.

**Risk:** High benefit, high impact items first.

**Notes:** Diagnostic & Tuning Pack required.

### SQL

```
WITH tasks AS (
  SELECT task_id, task_name, instance_id AS instance_number, execution_start,
execution_end
  FROM   dba_advisor_tasks
  WHERE  advisor_name = 'ADDM'
  AND    status = 'COMPLETED'
  AND    execution_end > SYSDATE - 7
)
SELECT t.instance_number, t.task_name,
       TO_CHAR(t.execution_end,'YYYY-MM-DD HH24:MI') AS end_time,
       r.rank, r.type, ROUND(r.benefit,2) AS benefit,
       ROUND(r.impact ,2) AS impact,
       SUBSTR(r.message,1,200) AS message_sample
FROM   tasks t
JOIN   dba_advisor_recommendations r ON r.task_id = t.task_id
ORDER  BY t.execution_end DESC, r.rank, r.impact DESC
FETCH FIRST 80 ROWS ONLY;
```

**Status:**

**Comment:**

## AWR IO by snapshot pairs (last hour)

**Purpose:** Read/write MB and throughput.

**Risk:** Sustained high throughput with waits suggests IO bottleneck.

**Notes:** Diagnostic & Tuning Pack required.

### SQL

```
WITH snaps AS (
  SELECT dbid, instance_number, snap_id, begin_interval_time, end_interval_time
  FROM   dba_hist_snapshot
  WHERE  end_interval_time > SYSDATE - 1/24
),
pairs AS (
  SELECT a.dbid, a.instance_number, a.snap_id AS begin_snap, b.snap_id AS
end_snap,
         a.end_interval_time AS begin_time, b.end_interval_time AS end_time,
         (b.end_interval_time - a.end_interval_time) * 86400 AS elapsed_s
  FROM   snaps a
  JOIN   snaps b ON b.dbid = a.dbid AND b.instance_number = a.instance_number
AND b.snap_id = a.snap_id + 1
),
stat AS (
  SELECT s.dbid, s.instance_number, s.snap_id, n.stat_name, s.value
  FROM   dba_hist_sysstat s
  JOIN   dba_hist_stat_name n
         ON n.dbid = s.dbid AND n.stat_id = s.stat_id
  WHERE  n.stat_name IN ('physical read total bytes','physical write total
bytes')
)
SELECT p.instance_number,
       TO_CHAR(p.begin_time,'YYYY-MM-DD HH24:MI') AS begin_time,
       TO_CHAR(p.end_time  ,'YYYY-MM-DD HH24:MI') AS end_time,
       ROUND( ( MAX(CASE WHEN se.stat_name='physical read total bytes'  THEN
se.value END)
              - MAX(CASE WHEN sb.stat_name='physical read total bytes'  THEN
sb.value END) ) / 1048576, 2) AS read_mb,
       ROUND( ( MAX(CASE WHEN se.stat_name='physical write total bytes' THEN
se.value END)
              - MAX(CASE WHEN sb.stat_name='physical write total bytes' THEN
sb.value END) ) / 1048576, 2) AS write_mb,
       ROUND( ( ( MAX(CASE WHEN se.stat_name='physical read total bytes'  THEN
se.value END)
                  - MAX(CASE WHEN sb.stat_name='physical read total bytes'  THEN
sb.value END) ) / NULLIF(p.elapsed_s,0) ) / 1048576, 3) AS read_MBps,
       ROUND( ( ( MAX(CASE WHEN se.stat_name='physical write total bytes' THEN
se.value END)
                  - MAX(CASE WHEN sb.stat_name='physical write total bytes' THEN
sb.value END) ) / NULLIF(p.elapsed_s,0) ) / 1048576, 3) AS write_MBps
FROM   pairs p
JOIN   stat se ON se.dbid = p.dbid AND se.instance_number = p.instance_number
AND se.snap_id = p.end_snap
```

```
JOIN   stat sb ON sb.dbid = p.dbid AND sb.instance_number = p.instance_number
AND sb.snap_id = p.begin_snap
           AND sb.stat_name = se.stat_name
GROUP  BY p.instance_number, p.begin_time, p.end_time, p.elapsed_s
ORDER  BY p.instance_number, p.end_time DESC;
```

**Status:**

**Comment:**

## AWR System events (last hour, User I/O)

**Purpose:** See top I/O waits with deltas.

**Risk:** High seconds_waited pinpoints heavy events.

**Notes:** Diagnostic & Tuning Pack required.

### SQL

```
SELECT e.instance_number, e.event_name,
       ROUND(SUM(e.time_waited_micro_delta)/1e6, 2) AS seconds_waited,
       SUM(e.total_waits_delta) AS waits
FROM   dba_hist_system_event e
JOIN   dba_hist_snapshot s
       ON s.dbid = e.dbid
      AND s.instance_number = e.instance_number
      AND s.snap_id = e.snap_id
WHERE  s.end_interval_time > SYSDATE - 1/24
AND    e.wait_class = 'User I/O'
GROUP  BY e.instance_number, e.event_name
ORDER  BY seconds_waited DESC
FETCH FIRST 15 ROWS ONLY;
```

**Status:**

**Comment:**

## Index depth check

**Purpose:** Catch deep B*trees, poor clustering, large invalids.

**Risk:** BLEVEL >= 4 on large indexes may need attention.

### SQL

```
WITH ix AS (
  SELECT owner, index_name, table_name, status, blevel, leaf_blocks,
clustering_factor, num_rows
  FROM   dba_indexes
  WHERE  index_type='NORMAL'
),
sz AS (
  SELECT owner, segment_name, ROUND(SUM(bytes)/1024/1024,2) AS size_mb
  FROM   dba_segments
  WHERE  segment_type='INDEX'
```

```
  GROUP  BY owner, segment_name
)
SELECT ix.owner, ix.index_name, ix.table_name, ix.status, ix.blevel,
ix.leaf_blocks,
       ix.clustering_factor, ix.num_rows, sz.size_mb
FROM   ix
LEFT   JOIN sz ON sz.owner=ix.owner AND sz.segment_name=ix.index_name
WHERE  (ix.status <> 'VALID' OR ix.blevel >= 4
        OR (ix.clustering_factor > (ix.num_rows*0.9) AND ix.num_rows > 100000))
ORDER  BY ix.blevel DESC, sz.size_mb DESC NULLS LAST
FETCH FIRST 200 ROWS ONLY;
```

**Status:**

**Comment:**

## Monitor Active Sessions (single instance)

**Purpose:** Detailed monitoring of active sessions across RAC instances, including SQL, waits, blocking, and programs.

**Risk:** High number of long active sessions indicates performance bottlenecks, blocking, or runaway SQL.

Notes: Single-instance view using V$SESSION and V$SQL.

```
set lines 180
COLUMN   ELAP      FORMAT  99999
COLUMN   WAIT_S    FORMAT  999999
COLUMN   BLK       FORMAT  9999
COLUMN   sid       FORMAT  9999
COLUMN   serial# heading ser# FORMAT 99999
COLUMN   sql_text   FORMAT  a40 truncate
COLUMN   event     FORMAT  a20 truncate
COLUMN   username   FORMAT  a10 truncate
COLUMN   program    FORMAT  a15 truncate
col inst for 9999
col spid for a6
col wait_s for 999999
col message for a85

SELECT s.inst_id inst, s.sid, s.serial#, s.username,
     s.last_call_et elap, s.event, s.seconds_in_wait wait_s,
     s.blocking_session blk, sq.sql_id, sq.plan_hash_value plan_hv,
     s.program, sq.sql_text
FROM   v$session s, v$sql sq
WHERE  s.sql_id = sq.sql_id
AND    s.inst_id = sq.inst_id
```

```
AND    s.status = 'ACTIVE'
AND    s.sql_child_number = sq.child_number
ORDER  BY s.username, s.sql_id;
```

**Status:**

**Comment:**

## Daily Tuning & Monitoring Pack — Additions

### System KPIs — 1-min & 15-min metrics

**Purpose:** Quick health snapshot: CPU, I/O, redo, PGA, parse, execs per sec.

**Risk:** Sustained high 'Host CPU Utilization', low 'Buffer Cache Hit Ratio', or high 'Redo Generated Per Sec' may indicate pressure or hot spots.

**Notes:** Uses V$SYSMETRIC; no Diagnostic Pack needed.

SQL

```
set lines 180
COLUMN   ELAP       FORMAT  99999
COLUMN   WAIT_S     FORMAT  999999
COLUMN   BLK        FORMAT  9999
COLUMN   sid        FORMAT  9999
COLUMN   serial# heading ser# FORMAT 99999
COLUMN   sql_text    FORMAT  a40 truncate
COLUMN   event      FORMAT  a20 truncate
COLUMN   username    FORMAT  a10 truncate
COLUMN   program     FORMAT  a15 truncate
col wait_s for 999999
col message for a85

SELECT s.sid, s.serial#, s.username,
     s.last_call_et elap, s.event, s.seconds_in_wait wait_s,
     s.blocking_session blk, q.sql_id, q.plan_hash_value plan_hv,
     s.program, q.sql_text
FROM   v$session s
JOIN   v$sql     q ON s.sql_id = q.sql_id AND s.sql_child_number = q.child_number
WHERE  s.status = 'ACTIVE'
AND    s.type   <> 'BACKGROUND'
ORDER  BY s.username, q.sql_id;
```

**Comment:**

## Top segments by logical reads (buffer gets)

**Purpose:** Find hot tables/indexes driving logical IO.

**Risk:** Single segment dominating reads suggests missing index or bad plan.

**Notes:** Good first stop before digging into SQL.

SQL

```
SELECT owner, object_name, object_type,
       SUM(value) AS buffer_gets
FROM   v$segment_statistics
WHERE  statistic_name = 'logical reads'
GROUP  BY owner, object_name, object_type
ORDER  BY buffer_gets DESC
FETCH FIRST 30 ROWS ONLY;
```

**Status:**

**Comment:**

## Top segments by physical reads

**Purpose:** Spot segments causing real disk IO.

**Risk:** High physical reads on a few segments -> storage hotspots.

SQL

```
SELECT owner, object_name, object_type,
       SUM(value) AS physical_reads
FROM   v$segment_statistics
WHERE  statistic_name = 'physical reads'
GROUP  BY owner, object_name, object_type
ORDER  BY physical_reads DESC
FETCH FIRST 30 ROWS ONLY;
```

**Status:**

**Comment:**

## Temp usage by session (single instance)

**Purpose:** Who is consuming TEMP space right now.

**Risk:** One or more sessions using large TEMP can starve others -> critical during peaks.

**Notes:** Join to SQL text for quick action.

SQL

```
SELECT s.sid, s.serial#, s.username, s.program,
    u.blocks * ts.block_size / 1024 / 1024 AS temp_mb,
    s.sql_id
FROM   v$sort_usage u
JOIN   v$session    s  ON s.saddr = u.session_addr
JOIN   dba_tablespaces ts ON ts.tablespace_name = u.tablespace
ORDER  BY temp_mb DESC;
```

**Comment:**

## File IO hot spots (per datafile)

**Purpose:** Identify files with highest read/write latency and throughput.

**Risk:** High single-file latency indicates storage or contention issues.

**Notes:** Uses V$FILEMETRIC_HISTORY if available, else V$FILESTAT_X views.

SQL

```
SELECT TO_CHAR(begin_time,'YYYY-MM-DD HH24:MI:SS') AS begin_time,
       file#,
       singleblkrds, readtim, writetim, phyrds, phywrts,
       ROUND(CASE WHEN phyrds=0 THEN NULL ELSE readtim/phyrds END,3)  AS
ms_per_read,
       ROUND(CASE WHEN phywrts=0 THEN NULL ELSE writetim/phywrts END,3) AS
ms_per_write
FROM   v$filemetric_history
ORDER  BY begin_time DESC, ms_per_read DESC
FETCH FIRST 200 ROWS ONLY;
```

**Status:**

**Comment:**

## Long operations (single instance)

**Purpose:** Track long-running ops (full scans, index builds, RMAN...).

**Risk:** Unexpected long ops can block windows and SLAs.

SQL

```
SELECT sid, serial#, opname, target, sofar, totalwork,
    ROUND(sofar*100/NULLIF(totalwork,0),2) AS pct,
    elapsed_seconds, time_remaining, sql_id
FROM   v$session_longops
```

```
WHERE  sofar > 0 AND totalwork > 0
ORDER  BY pct DESC, elapsed_seconds DESC;
```

**Comment:**

## Latch & mutex contention (top)

**Purpose:** Highlight latch/mutex contention hotspots.

**Risk:** High sleeps or wait time => concurrency/scalability issues.

### SQL

```
SELECT name, gets, misses, immediate_gets, immediate_misses, sleeps
FROM   v$latch
ORDER  BY sleeps DESC, misses DESC
FETCH FIRST 20 ROWS ONLY;

SELECT namespace, gets, misses, sleeps, wait_time
FROM   v$mutex_sleep
ORDER  BY wait_time DESC
FETCH FIRST 20 ROWS ONLY;
```

**Status:**

**Comment:**

## Enqueue waits summary

**Purpose:** Summarize critical enqueue classes (TX, TM, HW, CF...).

**Risk:** Elevated enqueue waits can halt workload (blocking).

### SQL

```
SELECT eq_type, req_reason, SUM(total_req#) AS total_reqs, SUM(cum_wait_time)
AS ms_waited
FROM   v$enqueue_statistics
GROUP  BY eq_type, req_reason
ORDER  BY ms_waited DESC;
```

**Status:**

**Comment:**

# Redo & Archive Daily Monitor — Single Instance (File System)

Purpose: Track redo log switch frequency, archive log volume, and related waits daily to avoid excessive switches, recovery risks, and storage pressure.

### 1) Redo Log Switches per Day (last 7 days)

SQL

```
SELECT trunc(first_time) log_date,
       COUNT(*) switches
FROM   v$log_history
WHERE  first_time > SYSDATE - 7
GROUP  BY trunc(first_time)
ORDER  BY log_date;
```

Notes: Use to trend daily switch counts and identify abnormal spikes.

### 2) Redo Log Switches by Hour (last 24 hours)

SQL

```
SELECT to_char(first_time, 'YYYY-MM-DD HH24') log_hour,
       COUNT(*) switches
FROM   v$log_history
WHERE  first_time > SYSDATE - 1
GROUP  BY to_char(first_time, 'YYYY-MM-DD HH24')
ORDER  BY log_hour;
```

Notes: Pinpoint peak hours and confirm switch frequency aligns with targets.

### 3) Archive Log Space per Day (last 7 days)

SQL

```
SELECT trunc(COMPLETION_TIME) log_date,
       ROUND(SUM(blocks*block_size)/1024/1024,2) AS mb_generated
FROM   v$archived_log
WHERE  COMPLETION_TIME > SYSDATE - 7
AND    standby_dest = 'NO'
GROUP  BY trunc(COMPLETION_TIME)
ORDER  BY log_date;
```

Notes: Daily archive volume helps plan FRA sizing and backup throughput.

### 4) Current Redo Log Files

SQL

```
SELECT group#, thread#, bytes/1024/1024 AS size_mb, status
FROM   v$log
ORDER  BY group#;
```

Notes: Validate size and count of redo groups match workload (typically >=3 groups).

### 5) Redo-related Waits Snapshot

SQL

```
SELECT event, total_waits, time_waited/100 AS time_waited_s
FROM   v$system_event
WHERE  event LIKE 'log file switch%'
   OR  event IN ('log file sync','log file parallel write')
ORDER  BY time_waited_s DESC;
```

Notes: If 'log file switch (archiving needed)' or 'checkpoint incomplete' appear, increase redo size and check archiver/I/O.

## Operational Thresholds & Risks

• Switch every 15–20 minutes during peak → OK (balanced).
• Switch < 5 minutes → Redo logs too small → Increase redo size (e.g., 512MB → 1GB or 1GB → 2GB) to reduce switches.
• Switch > 1 hour → Redo logs too large → Risk of long crash recovery; consider reducing size.
• Daily archive generation → Track trend to plan FRA sizing & backup windows (capacity and throughput).
• Risks if ignored: 'log file switch (archiving needed)', 'checkpoint incomplete', excessive 'log file sync', backup overrun, FRA pressure, longer MTTR.

## Daily Checklist (Operator Notes)

- Check hourly switches in last 24h; target 3–4 per hour at peak (≈15–20 min)
- Confirm no redo-related critical waits in Top waits.
- Record daily archive MB; compare to FRA free space and backup throughput.
- Validate redo group size/count; keep at least 3 groups on fast storage.